Python Essentials for Machine Learning Forecasting

Forecasting with Machine Learning

Natasia Villadsen

2026-11-02

Introduction

Today's Lecture

NumPy

Arrays, operations, subsetting, NaNs, random draws, lagging

pandas

Data frames, wrangling, time-indexed data

Machine learning

Definitions and scope

The forecasting pipeline

Data wrangling, model selection, evaluation

scikit-learn

OLS, LASSO, ridge, random forests, pipelines

Practical Information

- Colab is used in course instruction and exercises
- Exam must be handed in notebook (.ipynb) format
- Slides are .html-based to facilitate embedded code
    - Allows you to manipulate code directly in the slides
- .pdf versions are available on Brightspace
- “Up to you” slides provide small exercises

Why Python?

Python is the programming language used in this course.

- Large array of libraries for **data wrangling** and **machine learning**
- (Nearly) exclusive language of deep learning
- Aligns with your prerequisite courses
- Favored in both academia and industry
- Free and open-source

The Four Libraries

NumPy

Arrays and numerical tasks

pandas

Data frames and wrangling

scikit-learn

All non-deep-learning models

PyTorch

Deep learning

NumPy – Numerical Python

What is NumPy?

- Go-to library for numerical tasks – mean, logs, etc.
- Data is stored in **arrays**, say `arr`

Arrays general term for scalars, vectors, matrices and tensors:

$$\texttt{arr} \in \mathbb{R}, \quad \texttt{arr} \in \mathbb{R}^n, \quad \texttt{arr} \in \mathbb{R}^{n \times m}, \quad \texttt{arr} \in \mathbb{R}^{m_1 \times \cdots \times m_n}$$

> **Tip**
>
> If not quite confident in ***scalars, vectors, matrices, tensors***, refer to the Maths Brush-Up

Always import library with the standard alias,

Python Code ⟳ Start Over ▷ Run Code

```
1 import numpy as np
```

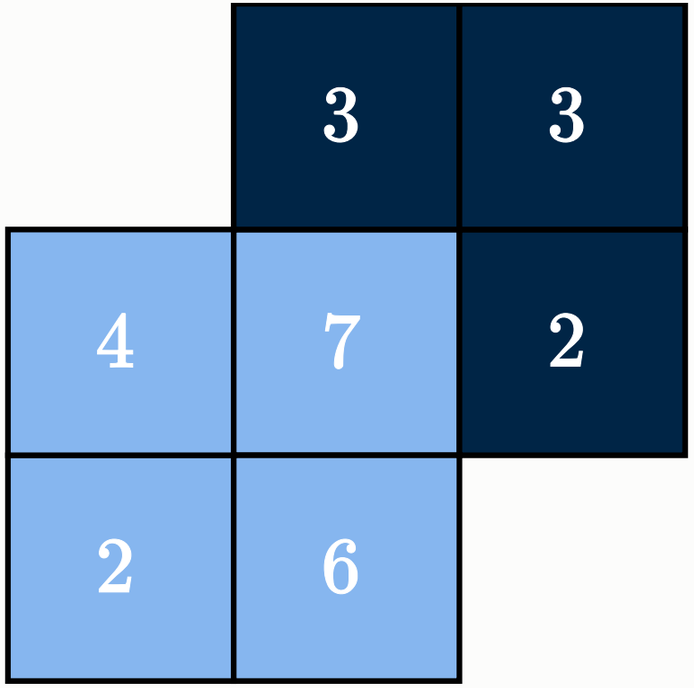
Defining Arrays

Python Code | Start Over | Run Code

```
arrS = np.array(4)                          # scalar
arrV = np.array([4, 7, 2])                  # vector
arrM = np.array([[4, 7, 2],
                 [6, 3, 3]])                # matrix
arrT = np.array([[[4, 7],
                  [2, 6]],
                  [[3, 3],
                  [1, 2]]])                 # tensor
print(arrS.shape, arrV.shape, arrM.shape, arrT.shape)
```

```
() (3,) (2, 3) (2, 2, 2)
```

Visualizing a 3D Array



Associated NumPy code: `arrT = np.array([[[4, 7], [2, 6]], [[3, 3], [1, 2]]])`

Functions on Arrays

Python Code | Start Over | Run Code

```
print(np.var(arrV))
print(np.log(arrM).round(2))
print(np.sum(arrT, axis=0))
```

```
4.222222222222222
[[1.39 1.95 0.69]
 [1.79 1.1  1.1 ]]
[[ 7 10]
 [ 3  8]]
```

Subsetting I – Basics

- Important to get right – differs from pandas
- Python is **zero-indexed**: `arr[row, col]`

Python Code | Start Over | Run Code

```
1 print(arrM)
2 print("One element:", arrM[1, 2])
3 print("First row:", arrM[0, :])
4 print("First column:", arrM[:, 0])
```

```
[[4 7 2]
 [6 3 3]]
One element: 3
First row: [4 7 2]
First column: [4 6]
```

Subsetting II – Intervals

Python Code

Start Over

Run Code

```
rng = np.random.default_rng(160923)
arrM = rng.standard_normal((100, 100))
arrMSub = arrM[0:20, 39:56]   # rows 1-20, columns 40-56
print(arrMSub.shape)
```

```
(20, 17)
```

> Zero-indexed, but the end of the interval is **excluded**

Subsetting III – Indices

Python Code | Start Over | Run Code

```
1 RowIdx = np.array([2, 4, 7])
2 ColIdx = np.array([9, 12, 56])
3 print(arrM[RowIdx, ColIdx].shape)          # wrong
4 print(arrM[np.ix_(RowIdx, ColIdx)].shape)
```

```
(3,)
(3, 3)
```

`arrM[RowIdx, ColIdx]` is wrong – it only selects three data points

Subsetting IV – Slicing

`::2` → every second index

Python Code | Start Over | Run Code

```
1 arrMEven = arrM[::2, :]
2 print(arrM.shape, arrMEven.shape)
```

```
(100, 100) (50, 100)
```

Up to you: Subsetting

Extract from `arrM`, every 4th column of rows 20 to 60 in increments of 3.

Exercise | Start Over | Show Solution | Run Code

```
1 arrMSub = ...
2 print(arrMSub.shape)
```

```
'ellipsis' object has no attribute 'shape'
```

NaNs I

Adding a number of NaNs to `arrM`,

Python Code | Start Over | Run Code

```
rng = np.random.default_rng(160923)
arrMNaN = arrM.copy()
arrMNaN[rng.choice(100, 10, replace=False), rng.choice(100, 10, replace=False)] = np.nan
print(np.isnan(arrMNaN).sum())
```

```
10
```

> `.copy()` is required – otherwise the original receives NaNs too

NaNs II

Determine whether NaNs are present and their count,

Python Code | Start Over | Run Code

```
arrMNaN0 = np.isnan(arrMNaN).any()      # Any NaNs present?
arrMNaNCount = np.isnan(arrMNaN).sum()  # Count the number of NaNs present
print(f"NaNs present: {arrMNaN0}. Count: {arrMNaNCount}")
```

```
NaNs present: True. Count: 10
```

We can handle NaNs by e.g. replacing them with values of our choice, e.g., 0 or the mean,

Python Code | Start Over | Run Code

```
arrMNaN0s = np.where(np.isnan(arrMNaN), 0, arrMNaN) # Replace NaNs with 0
arrMNaNMean = np.where(np.isnan(arrMNaN), np.nanmean(arrMNaN), arrMNaN) # Replace NaNs with mean
```

For time series, compute the mean on training data only

NaNs III - Drop Rows Containing NaNs

Python Code | Start Over | Run Code

```
1 arrMNoNaN   = arrMNaN[~np.isnan(arrMNaN).any(axis=1)] # Drop rows containing NaNs
2 print(arrMNaN.shape, arrMNoNaN.shape)
```

```
(100, 100) (90, 100)
```

1. `np.isnan(arrMNaN)` identifies the NaNs
2. `.any(axis=1)` – identifies any NaN in each row
3. `~` flips the selection to retain the clean rows

> Manipulating NaNs is dangerous – choose replacement values with care

Reshaping 1D Arrays

A 1D array has shape `(n,)`, but many functions expect a 2D array.

Reshape it into a column vector with `.reshape(-1, 1)`:

Python Code | Start Over | Run Code

```
rng = np.random.default_rng(160923)
arrV = rng.standard_normal(100)
print(arrV.shape)

arrV = arrV.reshape(-1, 1)
print(arrV.shape)
```

```
(100,)
(100, 1)
```

`-1` is a placeholder to let Python determine the number of rows, while `1` dictates one column

Pandas – Panel Data

What are pandas?

- Developed to match `R`'s (at the time) superior data handling
- Stores data in **data frames**. A single variable would be stored in a **series**
- Series are most commonly used for univariate targets, i.e., ys

Always import library with the standard alias,

Python Code ⟳ Start Over ▷ Run Code

```
1 import pandas as pd
```

Preliminary: Dictionaries

- Store **key–value pairs**: values are looked up by their key
- Defined with curly braces, `{key: value}`

Python Code | Start Over | Run Code

```
scores = {"OLS": 0.52, "LASSO": 0.48, "Ridge": 0.49}
print(scores["LASSO"])                # look up by key
scores["Random forest"] = 0.41        # add a new key
print(scores)
```

```
0.48
{'OLS': 0.52, 'LASSO': 0.48, 'Ridge': 0.49, 'Random forest': 0.41}
```

Keys are unique, usually strings; values can be anything.

We will use dictionaries for data frames, hyperparameter grids, and collecting results

Series

Defining a pandas series,

Python Code | Start Over | Run Code

```
1 series = pd.Series([4, 7, 2], index=["A", "B", "C"])
2 series
```

```
A    4
B    7
C    2
dtype: int64
```

The left-most column is the index. It can also be set afterwards with `series.index=[...]`

Data Frames

Defining a pandas data frame,

Python Code | Start Over | Run Code

```
rng = np.random.default_rng(160923)
df = pd.DataFrame({
    "Gaussian": rng.standard_normal(5),
    "Integer": rng.integers(0, 10, 5)
})
df
```

	Gaussian	Integer
0	0.268272	7
1	-1.026642	5
2	1.288725	6
3	0.648142	7
4	0.417430	6

Reading in Data

`pd.read_csv()` is an important function. Example with `toy_data.csv`,

Python Code | Start Over | Run Code

```
df = pd.read_csv("assets/data/toy_data.csv")
print(df.shape)
df.head()
```

```
(49, 4)
```

	Date	Description	Amount	Quantity
0	2026-02-18	B	41.577247	99
1	2026-02-17	C	69.127362	79
2	2026-02-16	D	49.547114	37
3	2026-02-15	E	49.910320	2
4	2026-02-14	F	37.917510	16

Tip

Use `df.to_csv("filename.csv")` to write a data frame to .csv

Inspecting Types

Python Code | Start Over | Run Code

```
1 df["Date"] = pd.to_datetime(df["Date"])
2 print(df.dtypes)
```

```
Date           datetime64[ns]
Description            object
Amount                float64
Quantity                int64
dtype: object
```

Also useful: `df.describe()`, `df.isna().sum()`, `df.dropna()`

Subsetting Data Frames I

Use `.iloc` for index-based subsetting, `.loc` for label-based subsetting,

Python Code Start Over Run Code

```
1 df.iloc[:3, :2]   # by position, rows 0-2 and columns 0-1, as in NumPy
```

	Date	Description
0	2026-02-18	B
1	2026-02-17	C
2	2026-02-16	D

Python Code Start Over Run Code

```
1 df.loc[:2, ["Description", "Amount"]]   # rows labelled 0-2, columns by label
```

	Description	Amount
0	B	41.577247
1	C	69.127362
2	D	49.547114

> As opposed to `.iloc` and NumPy, the end of `.loc` intervals is **included**

Subsetting Data Frames II

Subsetting by name alone (cannot subset rows simultaneously),

Python Code | Start Over | Run Code

```
1 df[["Amount", "Quantity"]].head(3)
```

	Amount	Quantity
0	41.577247	99
1	69.127362	79
2	49.547114	37

Subsetting by condition,

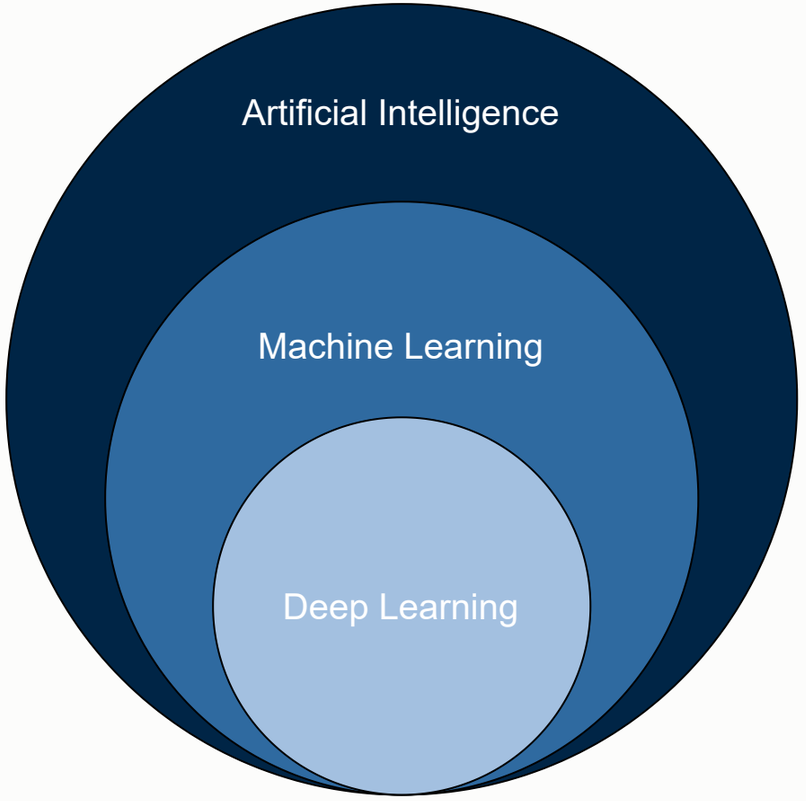
Python Code | Start Over | Run Code

```
1 df[df["Amount"] > 50].head(3)
```

	Date	Description	Amount	Quantity
1	2026-02-17	C	69.127362	79
6	2026-02-12	H	51.733712	72
7	2026-02-11	I	98.830141	30

Machine Learning

Layers of Intelligence



Deep learning is a subset of machine learning, which in turn is a subset of artificial intelligence.

Defining AI, Machine Learning, and Deep Learning

- Artificial intelligence is a programmed, possibly learned, set of rules
- Following Gu et al., 2020: The term **machine learning** encompasses:
    1. a diverse collection of high-dimensional models for statistical prediction, combined with
    2. so-called “regularization” methods for model selection and mitigation of overfit, and
    3. efficient algorithms for searching among a vast number of potential model specifications
- Following IBM, **deep learning** is a subset of machine learning driven by multilayered neural networks whose design is inspired by the structure of the human brain.

The Forecasting Pipeline

The Forecasting Pipeline

Pre-processing Explore distributions, check for NaNs and outliers, etc.

Lagging Construct predictors from past values only (time series forecasting)

Splitting Reserve the test set before further modelling

Standardizing Standardize on the training data, apply to both (all) data sets

Tuning Cross-validate hyperparameters on the training set

Fitting Fit the model on the merged training and validation sets with the selected hyperparameters

Evaluating Compare models on the held-out test set

The Forecasting Task

> We distinguish between classification and regression, categorical and numerical targets

CLASSIFICATION

- Categorical target
- Forecasting output: a class or class probabilities
- Errors: correct/incorrect
- Metrics: accuracy, ...

REGRESSION

- Numerical target
- Forecasting output: a continuous value
- Errors have a magnitude
- Metrics: MSE, MAE, R^2

Data Pre-Processing

Before modelling always inspect the raw data.

Python Code

```
df.head()           # first rows
df.describe()       # distributional summary
df.isna().sum()     # count missing values per column
df.dtypes           # check that data types are as expected
```

- Data pre-processing is essential in the forecasting pipeline
- We cannot dedicate due time to all the important steps in this lecture
- Instead, refer to Chapter 3 of Applied Predictive Modeling (Kuhn & Johnson, 2013)
- Code translated from `R` to Python is available in the Colab Notebook "Chapter 3 Translation"

Lagging I

$$Y_t = f(X_{t-1}) + \varepsilon_t$$

To forecast Y_t we may only use what was known at $t-1$, so the predictor is shifted back one period. Assume a random walk for simplicity – yet the concept holds for any predictor,

Python Code | Start Over | Run Code

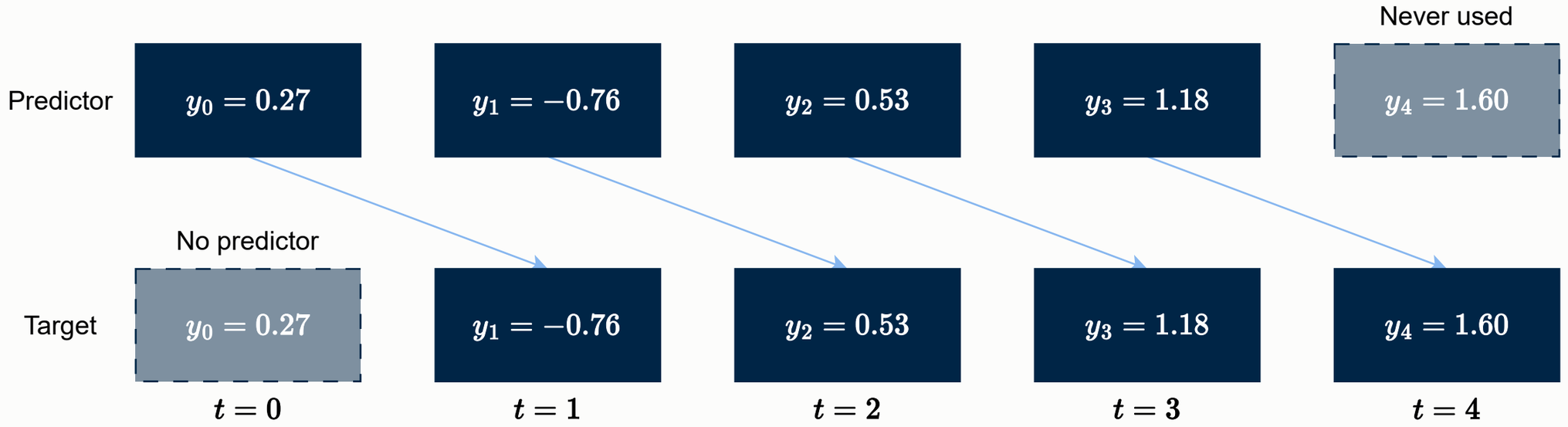
```
rng = np.random.default_rng(160923)
y = np.cumsum(rng.standard_normal(10))    # Random walk

y_lag = y[:-1]    # y at t-1: drop the last observation
y_t   = y[1:]     # y at t:   drop the first observation
print(y_lag.round(2))
print(y_t.round(2))
```

```
[ 0.27 -0.76  0.53  1.18  1.6   2.59  1.78  0.42 -1.16]
[-0.76  0.53  1.18  1.6   2.59  1.78  0.42 -1.16 -0.2 ]
```

Exercises 16 and 17 pertain to lagging in pandas with `.shift(1)`

Lagging II



Example of lagging a time series by one period. The top row is the original series, the bottom row is the lagged series.

- Each arrow is a forecast in the random walk: standing at $t - 1$ with y_{t-1}, predict y_t.
- `y[:-1]` is the top row without its last box, `y[1:]` the bottom row without its first box.

Sample Splitting

In cross-sectional modeling, data is usually split randomly

In **time series modeling**, data must be split chronologically, such as,

Python Code

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, train_size=0.8, shuffle=False
)
```

Standardization

Z-score standardization: Subtract the mean, divide by the standard deviation,

$$z_t = \frac{x_t - \hat{\mu}}{\hat{\sigma}}$$

Yields z_t with zero mean and unit variance.

STANDARDIZE

- Shrinkage models (LASSO, ridge, elastic net)
  - Penalty not scale-invariant
- Neural networks
  - Gradient descent is sensitive to scale

STANDARDIZATION UNNECESSARY

- Tree-based models (random forests)
  - Splits are scale-invariant
- OLS
  - Coefficients scale with data

> Fit the scaler on **training data only**, then apply to the test set
> Fitting on the full sample leaks future distributional information

Standardization: Code

Python Code

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_sc = scaler.fit_transform(X_train)  # fit + transform on train
X_test_sc = scaler.transform(X_test)        # transform only on test
```

The Bias-Variance Trade-Off

The expected squared forecast error decomposes as

$$\mathrm{E}\left[(\hat{y} - y)^2\right] = \underbrace{\mathrm{Bias}^2}_{\text{underfitting}} + \underbrace{\mathrm{Variance}}_{\text{overfitting}} + \underbrace{\sigma^2}_{\text{irreducible}}$$

HIGH BIAS

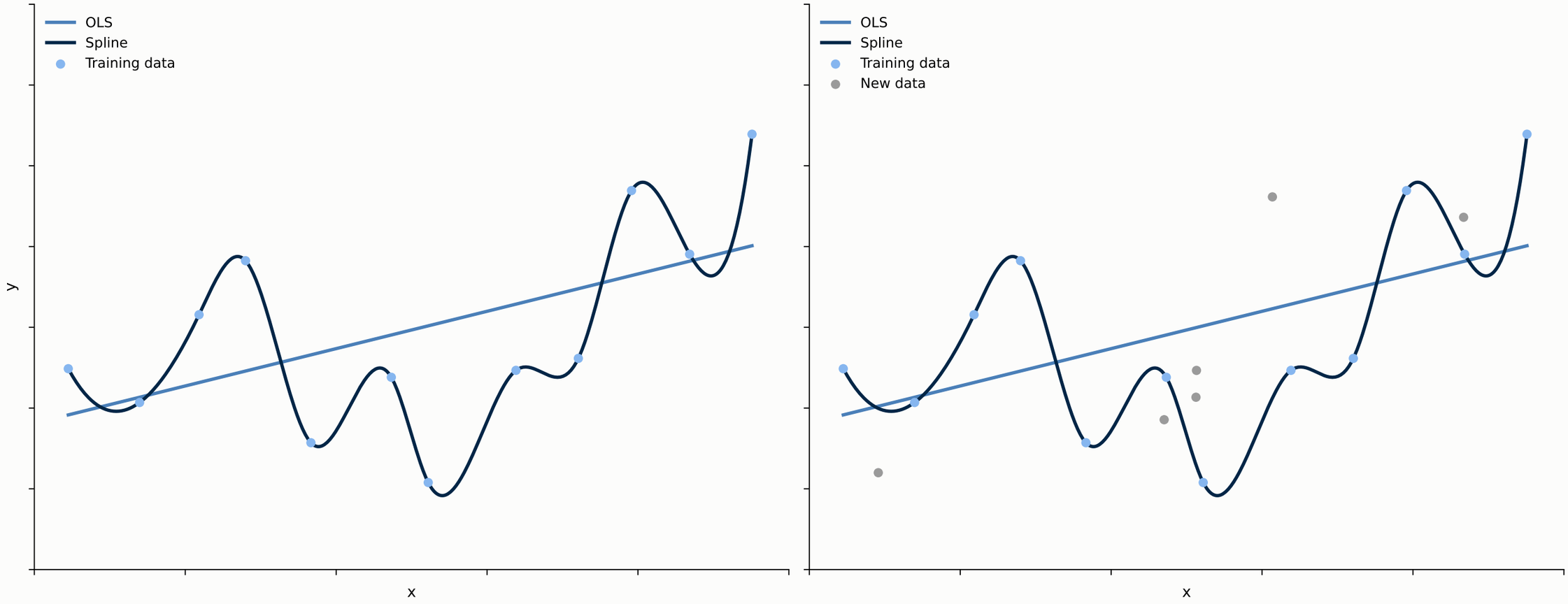
- Model too simple
- Misses systematic patterns
- Train error $\approx$ test error, both large

HIGH VARIANCE

- Model too complex/flexible
- Model fits to noise rather than signal
- Train error $\ll$ test error

Regularized models, e.g., LASSO, trade off bias for variance, while ensembles such as random forests reduce variance

Bias-variance Trade-Off – An Extreme Example



MSE OLS (left-hand side): 0.262, MSE spline (left-hand side): 0.000, MSE OLS (right-hand side): 0.209, MSE spline (right-hand side): 0.394.

scikit-learn

What is scikit-learn?

Primary modeling library covering a large array of models:

- Linear regression (OLS)
- Shrinkage models (LASSO, ridge, elastic net)
- Tree-based models (CART, random forests, gradient boosting…)
- Support vector regression/machines

> Advantage: unified and streamlined syntax – `fit` $\rightarrow$ `predict` $\rightarrow$ `score` (R^2_{OOS})

Importing

Generally import functions as needed, e.g.,

Python Code | Start Over | Run Code

```
from sklearn.linear_model import LinearRegression, Lasso, Ridge, LassoCV, RidgeCV
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.model_selection import TimeSeriesSplit
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
from sklearn.pipeline import Pipeline
```

> Import from `sklearn` and not `scikit-learn`

Practical Example: Forecasting US Inflation

Quarterly US macroeconomic data, 1959Q1–2009Q3, from `statsmodels`

- Target: inflation, `infl`
- Predictors: lagged inflation, unemployment and the T-bill rate

Python Code | Start Over | Run Code

```
import statsmodels.api as sm
df = sm.datasets.macrodata.load_pandas().data
df_sub = df[["infl", "unemp", "tbilrate"]].to_numpy()   # (T, 3)
infl = df_sub[:, 0]
print(df_sub.shape)
df[["infl", "unemp", "tbilrate"]].head(3)
```

```
(203, 3)
```

	infl	unemp	tbilrate
0	0.00	5.8	2.82
1	2.34	5.1	3.08
2	2.74	5.3	3.82

Pre-Processing Inflation Data

Python Code | Start Over | Run Code

```python
X = df_sub[:-1]     # (infl, unemp, tbilrate) at t-1: drop final observation
y = infl[1:]        # infl at t: drop first observation

X_train, X_test, y_train, y_test = train_test_split(
    X, y, train_size=0.8, shuffle=False
)
print(f"Train dimension X: {X_train.shape}, Test dimension X: {X_test.shape}")
print(f"Train dimension y: {y_train.shape}, Test dimension y: {y_test.shape}")

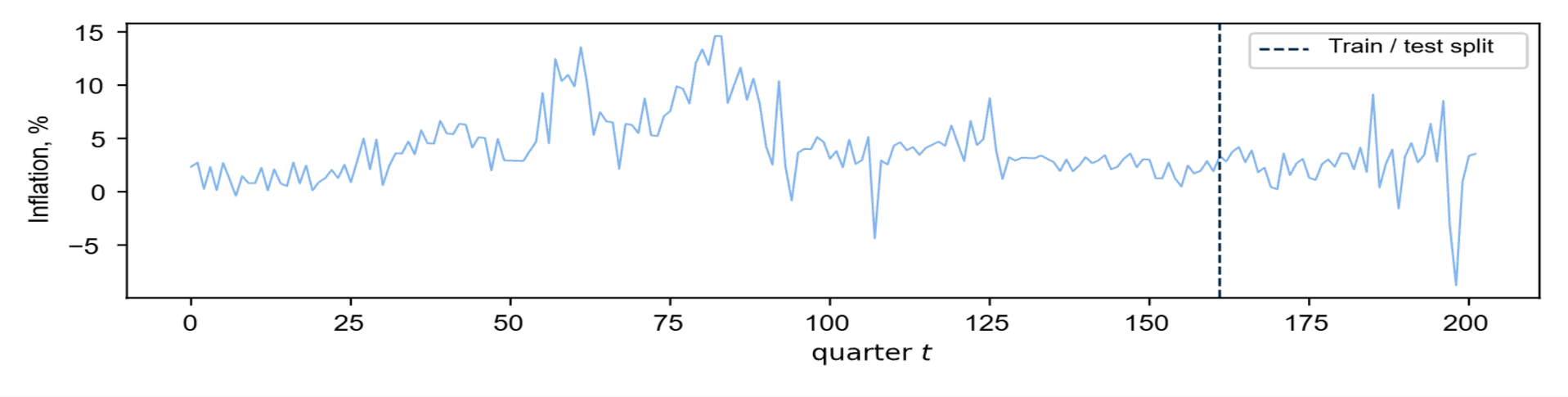
scaler     = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)
```

```
Train dimension X: (161, 3), Test dimension X: (41, 3)
Train dimension y: (161,), Test dimension y: (41,)
```

Visualizing the Target

Python Code | Start Over | Run Code

```
1 import matplotlib.pyplot as plt
2 fig, ax = plt.subplots(figsize=(8, 2.5))
3 ax.plot(range(len(y)), y, lw=0.8, color="#86b6ef")
4 ax.axvline(len(y_train), color="#002546", lw=1, ls="--", label="Train / test split")
5 ax.set_xlabel("quarter $t$"); ax.set_ylabel("Inflation, %"); ax.legend(fontsize=9)
6 plt.tight_layout(); plt.show()
```



Refer to the Colab Notebook "matplotlib documentation" and the examples to learn about plotting with matplotlib and its pyplot module

The Random Walk Benchmark

- The random walk is commonly used as benchmark in time series forecasting
- Predicts the next value as the previous value,

$$\hat{y}_t = y_{t-1}$$

Python Code | Start Over | Run Code

```python
y_pred_rw = np.concatenate([[y_train[-1]], y_test[:-1]])   # previous value
mse_rw = mean_squared_error(y_test, y_pred_rw)
print(f"Random walk MSE = {mse_rw:.4f}")
```

```
Random walk MSE = 13.8971
```

Ordinary Least Squares (OLS)

- *The* linear regression model
- Determine how the target variable y depends linearly on a set of exogenous variables, X,
  - Compute the coefficients, β, by minimizing the sum of squared residuals,

$$\hat{\beta}_{\text{OLS}} = \arg\min_{\beta} \left\{ \sum_{t=1}^{T} (y_t - x_t^\top \beta)^2 \right\}$$

> Intuitive and interpretable, yet possibly challenged by heteroskedasticity and serial correlation

Linear Regression (OLS) in Code

Python Code | Start Over | Run Code

```
ols = LinearRegression()
ols.fit(X_train, y_train)
y_pred_ols = ols.predict(X_test)
mse_ols = mean_squared_error(y_test, y_pred_ols)
print(f"OLS MSE = {mse_ols:.4f}")
```

```
OLS MSE = 9.9284
```

OLS in scikit-learn

Implementation on previous slide follows the standard scikit-learn blueprint,

Initialize model object

`ols = LinearRegression()` assigns a linear regression object to `ols`

Fitting/training model

`.fit(X_train, y_train)` accepts the training covariates and target and fits/trains the model on this data

Predicting

`.predict(X_test)` uses the out-of-sample covariates to generate predictions of the target

Evaluating

`.score(X_test, y_test)` returns the R^2 against the test mean

> The mean squared error is a common metric for model evaluation, but others exist as well

Up to you: OLS

Compute the **in-sample** fitted values from `ols` and the associated training MSE. The relevant data and fitted model are stored in the memory already, so you can proceed from here.

Exercise | Start Over | Show Solution | Run Code

```
1 y_pred_ols_train = ...
2 mse_ols_train = ...
3 print(f"Train MSE = {mse_ols_train:.4f}")
```

```
unsupported format string passed to ellipsis.__format__
```

Penalized Regression Models I

- Three primary penalized regression models; i) LASSO, ii) ridge, and iii) elastic net
- Each adds a penalty term to the ordinary least squares (OLS) objective function of ℓ_1, ℓ_2, and both, respectively. The models are defined as,

$$\text{LASSO:} \quad \hat{\beta}_{\text{LASSO}} = \arg\min_{\beta} \left\{ \sum_{t=1}^{T} (y_t - x_t^\top \beta)^2 + \lambda \sum_{j=1}^{p} |\beta_j| \right\}$$

$$\text{Ridge:} \quad \hat{\beta}_{\text{Ridge}} = \arg\min_{\beta} \left\{ \sum_{t=1}^{T} (y_t - x_t^\top \beta)^2 + \lambda \sum_{j=1}^{p} \beta_j^2 \right\}$$

$$\text{Elastic net:} \quad \hat{\beta}_{\text{ENet}} = \arg\min_{\beta} \left\{ \sum_{t=1}^{T} (y_t - x_t^\top \beta)^2 + \lambda \left[\rho \sum_{j=1}^{p} |\beta_j| + (1-\rho) \sum_{j=1}^{p} \beta_j^2 \right] \right\}$$

where $\lambda \geq 0$ governs the penalty strength and $\rho \in [0, 1]$ the mix of ℓ_1 and ℓ_2.

> **Tip**
>
> In `sklearn`, the penalty strength is called `alpha`. For ridge, `alpha` $= \lambda$. For LASSO, `alpha` $= \lambda/(2T)$

Penalized Regression Models II: Selection and Shrinkage

- The **LASSO** is a variable selection method, as, by virtue of the ℓ_1 penalty, coefficients can be set to exactly zero
    - Among strongly correlated predictors it tends to keep one and drop the others
- **Ridge** shrinks all coefficients towards zero but sets none of them exactly to zero
- The **elastic net** selects and shrinks, with ρ governing the mix
    - Correlated predictors tend to be kept or dropped together
- The penalty strength λ and the mixing weight ρ are hyperparameters, tuned by time-series cross-validation

> Penalized regression models trade off bias for variance relative to OLS

Non-Optimized LASSO and Ridge

Python Code | Start Over | Run Code

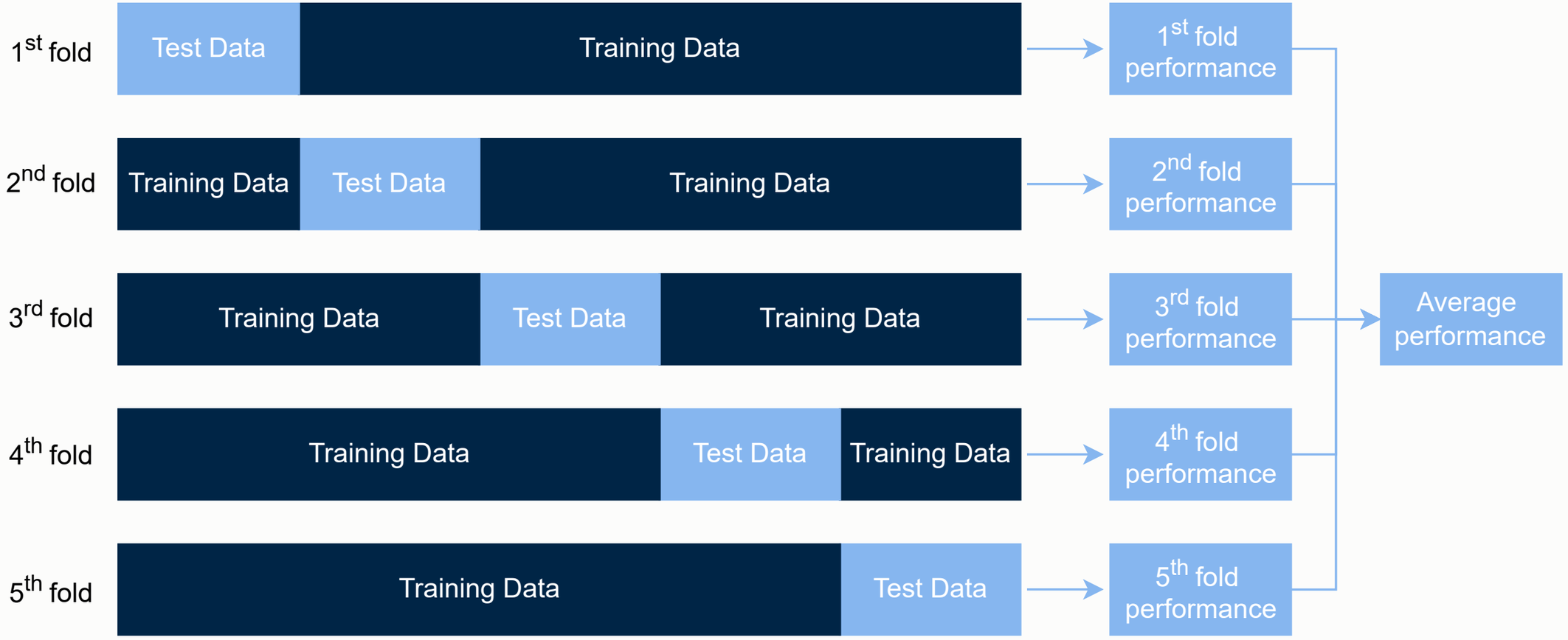
```python
lasso = Lasso(alpha=0.05).fit(X_train_sc, y_train)
ridge = Ridge(alpha=1.0).fit(X_train_sc, y_train)

y_pred_lasso = lasso.predict(X_test_sc)
y_pred_ridge = ridge.predict(X_test_sc)

mse_lasso = mean_squared_error(y_test, y_pred_lasso)
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
print(f"LASSO MSE = {mse_lasso:.4f}")
print(f"Ridge MSE = {mse_ridge:.4f}")
```

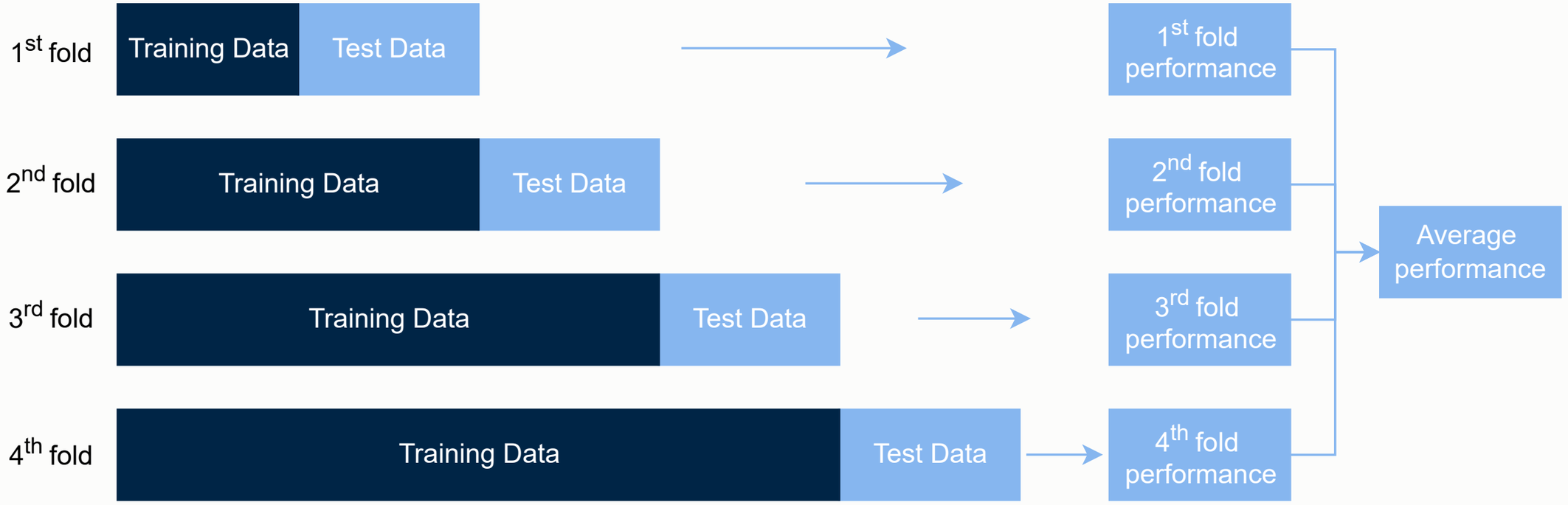
```
LASSO MSE = 9.7992
Ridge MSE = 9.8909
```

Cross-Sectional Cross-Validation



Cross-sectional cross-validation depicted for a 5-fold split.

Time Series Cross-Validation



Time series cross-validation depicted for a 4-fold split.

Cross-Validated LASSO and Ridge

`RidgeCV` selects by R^2 unless `scoring` is set, so MSE is passed explicitly

Python Code | Start Over | Run Code

```python
lasso = LassoCV(cv=TimeSeriesSplit()).fit(X_train_sc, y_train)
ridge = RidgeCV(cv=TimeSeriesSplit(),
                scoring="neg_mean_squared_error").fit(X_train_sc, y_train)

y_pred_lasso = lasso.predict(X_test_sc)
y_pred_ridge = ridge.predict(X_test_sc)

mse_lasso = mean_squared_error(y_test, y_pred_lasso)
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
print(f"LASSO alpha = {lasso.alpha_:.4f}, MSE = {mse_lasso:.4f}")
print(f"Ridge alpha = {ridge.alpha_:.4f}, MSE = {mse_ridge:.4f}")
```

```
LASSO alpha = 1.0172, MSE = 9.5005
Ridge alpha = 10.0000, MSE = 9.5995
```

The scaler is fitted before cross-validation, so there might be leakage of future information into the model

Pipelines handle scaling intrinsically

Pipelines: Chaining the Steps I

A scikit-learn `Pipeline` chains preprocessing and model in **one object**

Define
`(name, object)` tuples; the last step is the model

→

`.fit()`
Fits the scaler, transforms, then fits the model

→

`.predict()`
Applies the fitted scaler, then predicts: no leakage

Python Code | Start Over | Run Code

```
lasso_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model",  Lasso(alpha=0.05))
])
lasso_pipe.fit(X_train, y_train)                # raw, unscaled data
y_pred_pipe = lasso_pipe.predict(X_test)

# Continued on next slide
```

Pipelines: Chaining the Steps II

Python Code | Start Over | Run Code

```
# Continued from previous slide

# Step-by-step equivalent, using the scaled data from before
y_pred_step = Lasso(alpha=0.05).fit(X_train_sc, y_train).predict(X_test_sc)
print(f"Pipeline  MSE = {mean_squared_error(y_test, y_pred_pipe):.4f}")
print(f"Step-wise MSE = {mean_squared_error(y_test, y_pred_step):.4f}")
```

```
Pipeline  MSE = 9.7992
Step-wise MSE = 9.7992
```

Pipelines and Cross-Validation

In cross-validation the scaler must be **re-fitted within every training fold**.

The pipeline does this automatically,

Python Code | Start Over | Run Code

```
pipe = Pipeline([("scaler", StandardScaler()), ("model", Lasso())])
search = GridSearchCV(pipe,
                      param_grid={"model__alpha": [0.001, 0.01, 0.1, 1.0]},
                      cv=TimeSeriesSplit(n_splits=5),
                      scoring="neg_mean_squared_error")
search.fit(X_train, y_train)
y_pred_cv = search.predict(X_test)
print(f"Tuned LASSO MSE = {mean_squared_error(y_test, y_pred_cv):.4f} "
      f"with optimal alpha = {search.best_params_['model__alpha']:.4f}")
```

```
Tuned LASSO MSE = 9.5055 with optimal alpha = 1.0000
```

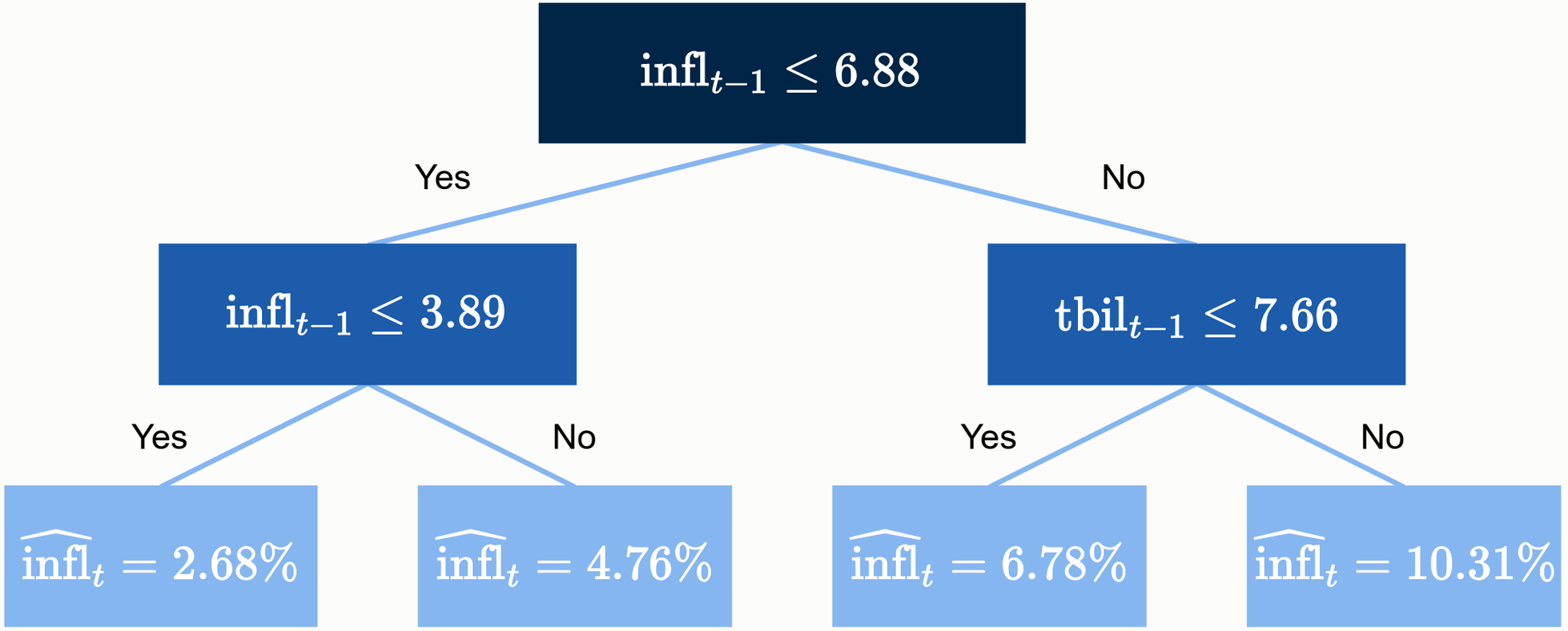
Decision Trees

- A tree forecasts by splitting the training data into **nodes** with yes/no questions about one predictor at a time, e.g., "is lagged inflation ≤ 6.88?"
- Nodes that are not split further are the **leaves**
- The forecast of leaf k is the mean of the training targets that end up in it,

$$\hat{y}_k = \frac{1}{n_k} \sum_{t \in \mathcal{L}_k} y_t$$

- where $\mathcal{L}_k$ is the set of training observations in leaf k, and n_k is their number
- A new observation is forecast by its associated leaf, assigned via the decision path

Visualizing a Decision Tree



The values depicted on the tree stem from a single run of a CART regressor on the data set in our example.

Decision Trees: Growing a Tree

- A split asks "is $x_{t,j} \leq s$?", with $x_{t,j}$ predictor j in observation t and s the cut point
- Each side of the node is forecast by its own mean target, $\bar{y}_{\text{left}}$ and $\bar{y}_{\text{right}}$, giving the squared error

$$\text{SSE}(j, s) = \sum_{t:\ x_{t,j} \leq s} \left(y_t - \bar{y}_{\text{left}}\right)^2 + \sum_{t:\ x_{t,j} > s} \left(y_t - \bar{y}_{\text{right}}\right)^2$$

- The pair (j, s) with the smallest SSE is chosen. We repeat in each new node until a stopping rule is met

> Deep trees fit the training data closely but have high variance – a single tree is unstable

Random Forests

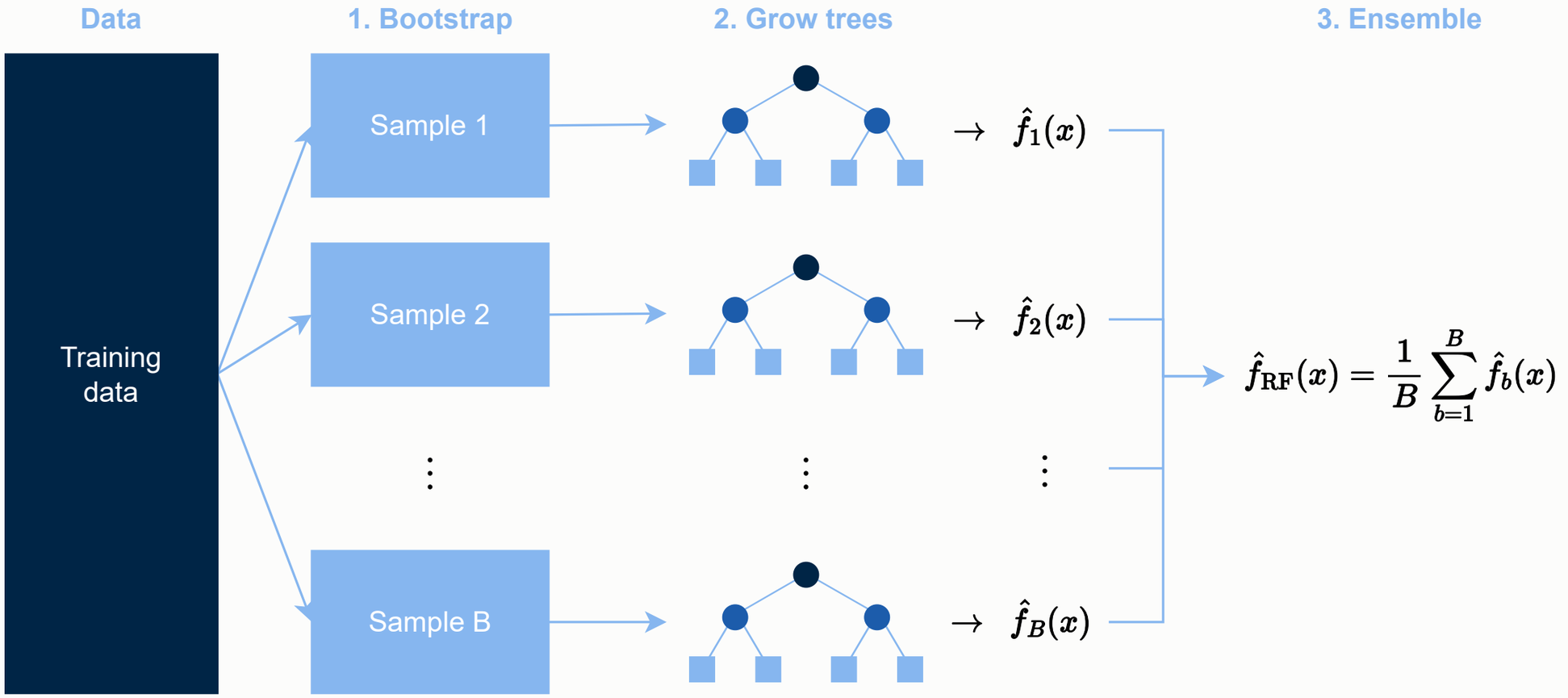
Reduce the variance of trees by **averaging** many trees grown on different bootstrap samples,

1	**Bootstrap**	Draw B samples with replacement from the training data
2	**Grow**	Fit a deep tree on each sample, using only $m < p$ randomly chosen predictors at each split, decorrelating the trees
3	**Average**	The forecast is the average over the B trees

$$\hat{f}_{\mathrm{RF}}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{f}_b(x)$$

> Forecasts are averages of training targets, so a forest cannot predict outside the range of the targets

Random Forests Visualized



Random Forests in Code

Python Code | Start Over | Run Code

```
rf = RandomForestRegressor(n_estimators=200, max_depth=4, random_state=160923)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
mse_rf = mean_squared_error(y_test, y_pred_rf)
print(f"Random forest MSE = {mse_rf:.4f}")
```

```
Random forest MSE = 9.9475
```

> Random forests are scale-invariant – no standardization required

Comparing Results

Python Code | Start Over | Run Code

```python
results = {
    "RW":            mse_rw,
    "OLS":           mse_ols,
    "LASSO (CV)":    mse_lasso,
    "Ridge (CV)":    mse_ridge,
    "Random forest": mse_rf,
}
for name, mse in results.items():
    print(f"{name:20s}  MSE = {mse:.4f}")
```

```
RW                   MSE = 13.8971
OLS                  MSE = 9.9284
LASSO (CV)           MSE = 9.5005
Ridge (CV)           MSE = 9.5995
Random forest        MSE = 9.9475
```

References

Bergmann, D. (2025). *What is deep learning?* IBM. https://www.ibm.com/think/topics/deep-learning

Gu, S., Kelly, B. and Xiu, D. (2020). Empirical asset pricing via machine learning. *Review of Financial Studies*, 33(5), 2223–2273.

Kuhn, M. and Johnson, K. (2013). *Applied Predictive Modeling*. Springer.

Appendix

Expanding and Rolling Windows

So far, the model was estimated once and used for the whole test sample.

A real-time forecaster would generally re-estimate as new data arrives,

1	**Estimate**	Fit the model on rows before t only
2	**Forecast**	Predict row t
3	**Move on**	Go to row $t+1$ and repeat

> Expanding: fit on all rows before t. Rolling: fit on the last W rows before t

Expanding and Rolling Windows: Code

- We fit OLS with expanding and rolling window estimation
- `X[:t]` is rows $0, \ldots, t-1$ (the end is excluded)
- Row t of `X` holds information from $t-1$, so no future data enters the fit

Python Code | Start Over | Run Code

```python
n_train = len(y_train)      # first test row
window = 80                 # rolling window length W

fc_exp, fc_rol = [], []
for t in range(n_train, len(y)):
    ols = LinearRegression().fit(X[:t], y[:t])                      # expanding
    fc_exp.append(ols.predict(X[t:t + 1])[0])

    ols = LinearRegression().fit(X[t - window:t], y[t - window:t])    # rolling
    fc_rol.append(ols.predict(X[t:t + 1])[0])
```

Comparing the Schemes

Python Code | Start Over | Run Code

```
print(f"Fixed     MSE = {mse_ols:.4f}")
print(f"Expanding MSE = {mean_squared_error(y_test, fc_exp):.4f}")
print(f"Rolling   MSE = {mean_squared_error(y_test, fc_rol):.4f}")
```

```
Fixed     MSE = 9.9284
Expanding MSE = 9.8209
Rolling   MSE = 8.4264
```

Fixed: one estimation (this lecture). Expanding and rolling: one estimation per test row